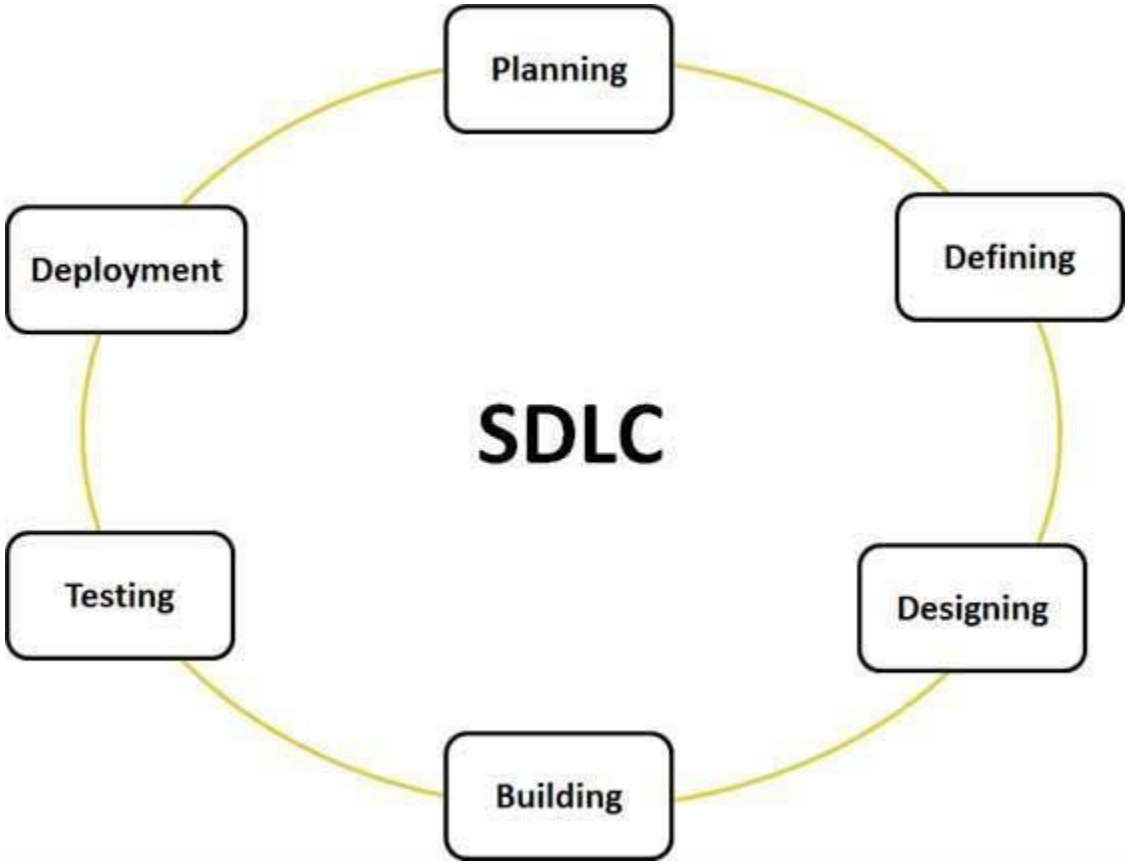
HCI IN THE SOFTWARE PROCESS

Software Engineering

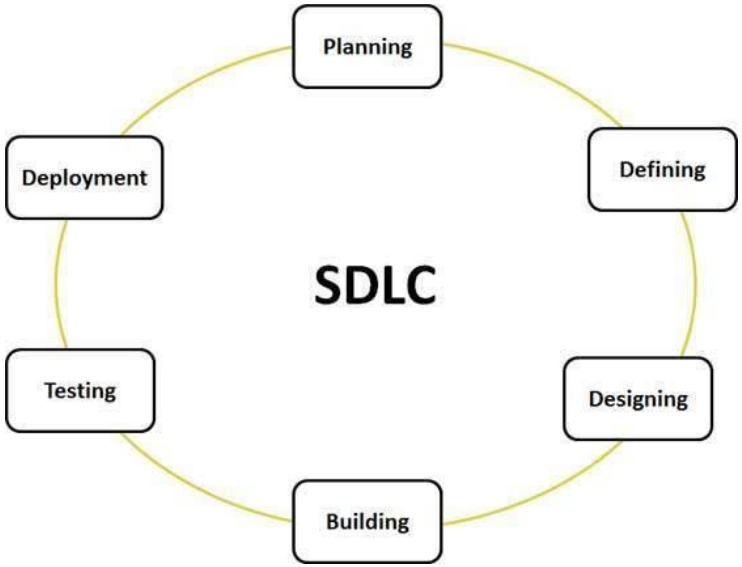
- Software engineering is the discipline for understanding the software design process, or life cycle (SDLC)
- Designing for usability occurs at all stages of the life cycle, not as a single isolated activity

Software lifecycle

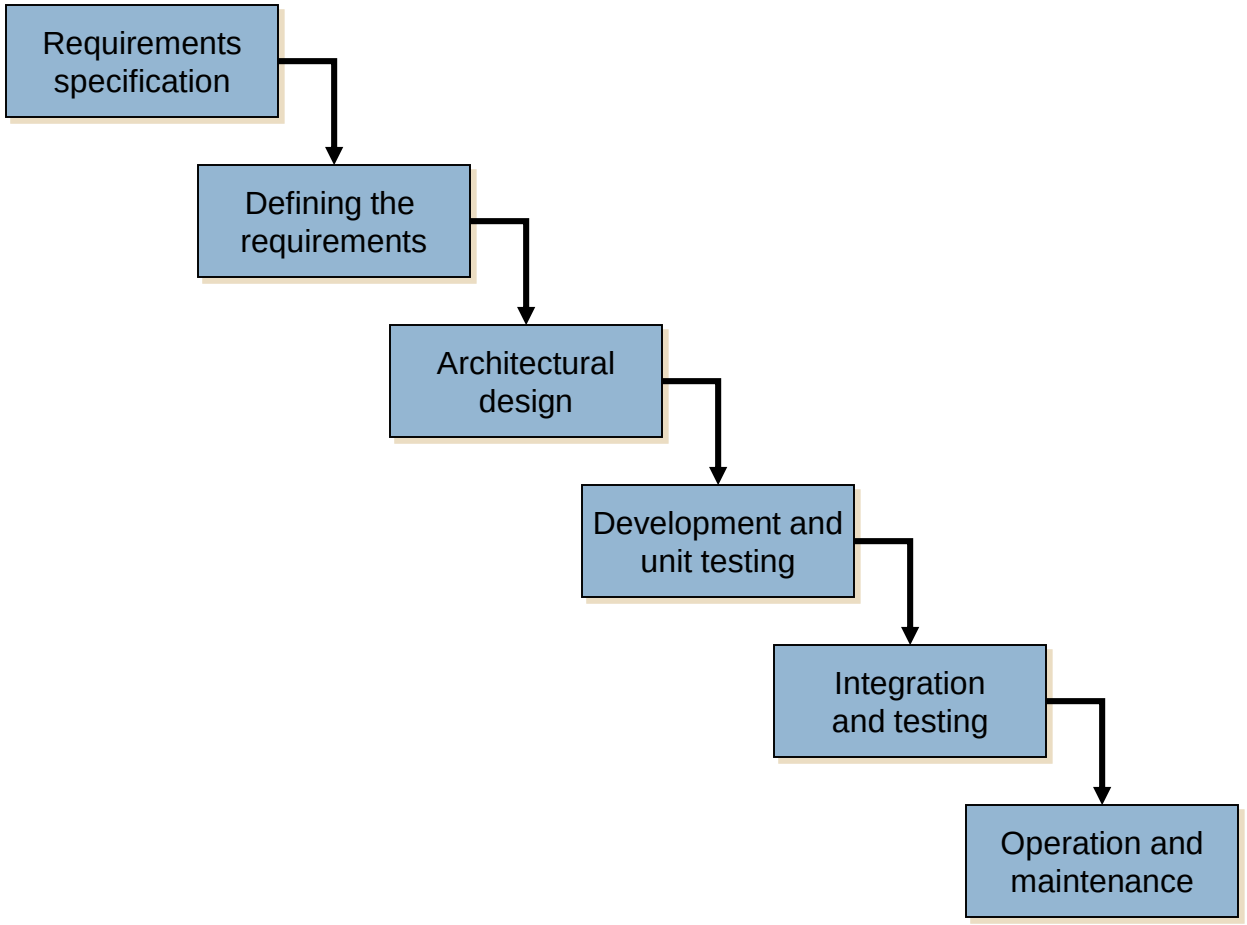


6 phases in software lifecycle

- Requirements specification
- Defining the requirements
- Architectural design
- Development and unit testing
- Integration and testing
- Operation and maintenance



The software lifecycle:The waterfall model



1. Requirement specification

- Designer and user (customer) try capture what the system is expected to provide
- It can be expressed in natural language
- or more precise languages, such as a task analysis would provide
- The requirements are collected using a number of practices as given
  - Studying the existing or obsolete system and software,
  - Conducting interviews of users and developers,
  - Referring to the database or
  - Collecting answers from the questionnaires.

2. Defining the requirements

- The next step is to clearly define and document the product requirements and get them approved from the customer or the market analysts.
- This is done through SRS. Software Requirement Specification document
- It consists of all the product requirements to be designed and developed during the project life cycle.

3. Architectural design

- High-level description of how the system will provide the services
- How it satisfy both functional and non functional requirements
- SRS is the reference for product architects to come out with the best architecture for the product to be developed. Based on the requirements specified in SRS, the best design approach is selected for the product and documented in a DDS - Design Document Specification.

4. Development and unit testing

- The programming code is generated as per DDS during this stage.
- If the design is performed in a detailed and organized manner, code generation can be accomplished without much hassle.
- Developers have to follow the coding guidelines defined by their organization and programming tools like compilers, interpreters, debuggers etc are used to generate the code.

5. Integration and testing

- This stage of SDLC is involved in the integration of software with outer world entities.
- Software may need to be integrated with the libraries, databases and other program(s).
- Also, during this stage, the product defects are reported, tracked, fixed and retested, until the product reaches the quality standards defined in the SRS.

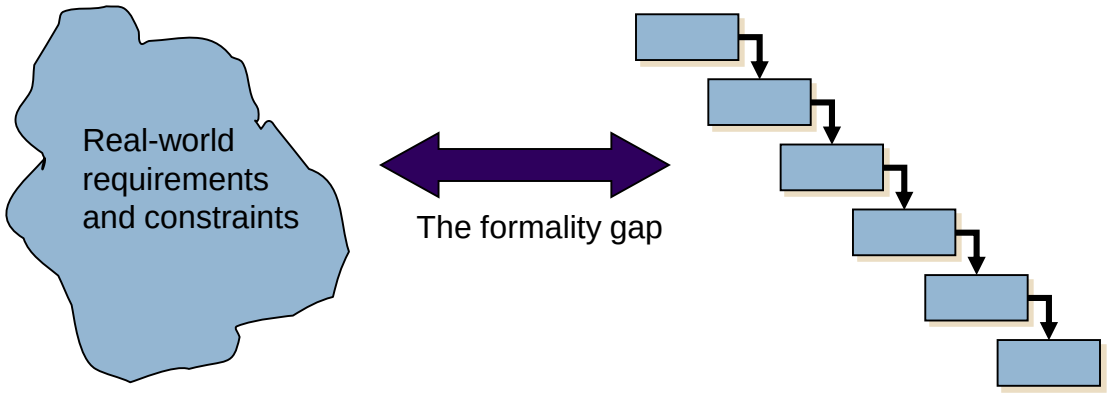
6. Operation and maintenance

- This phase confirms the software operation in terms of more efficiency and less errors.
- If required, the users are trained on, or aided with the documentation on how to operate the software and how to keep the software operational.

Verification and validation

- Each step contains verification and validation
- Verification:
  - Correct/Consistent representation of previous stage
  - It should produce correct results for correct inputs
  - Designing the right product
- Validation:
  - Check product to ensure results are conform requirements
  - It should produce error for incorrect inputs
  - Designing the product right

Verification and validation



The formality gap

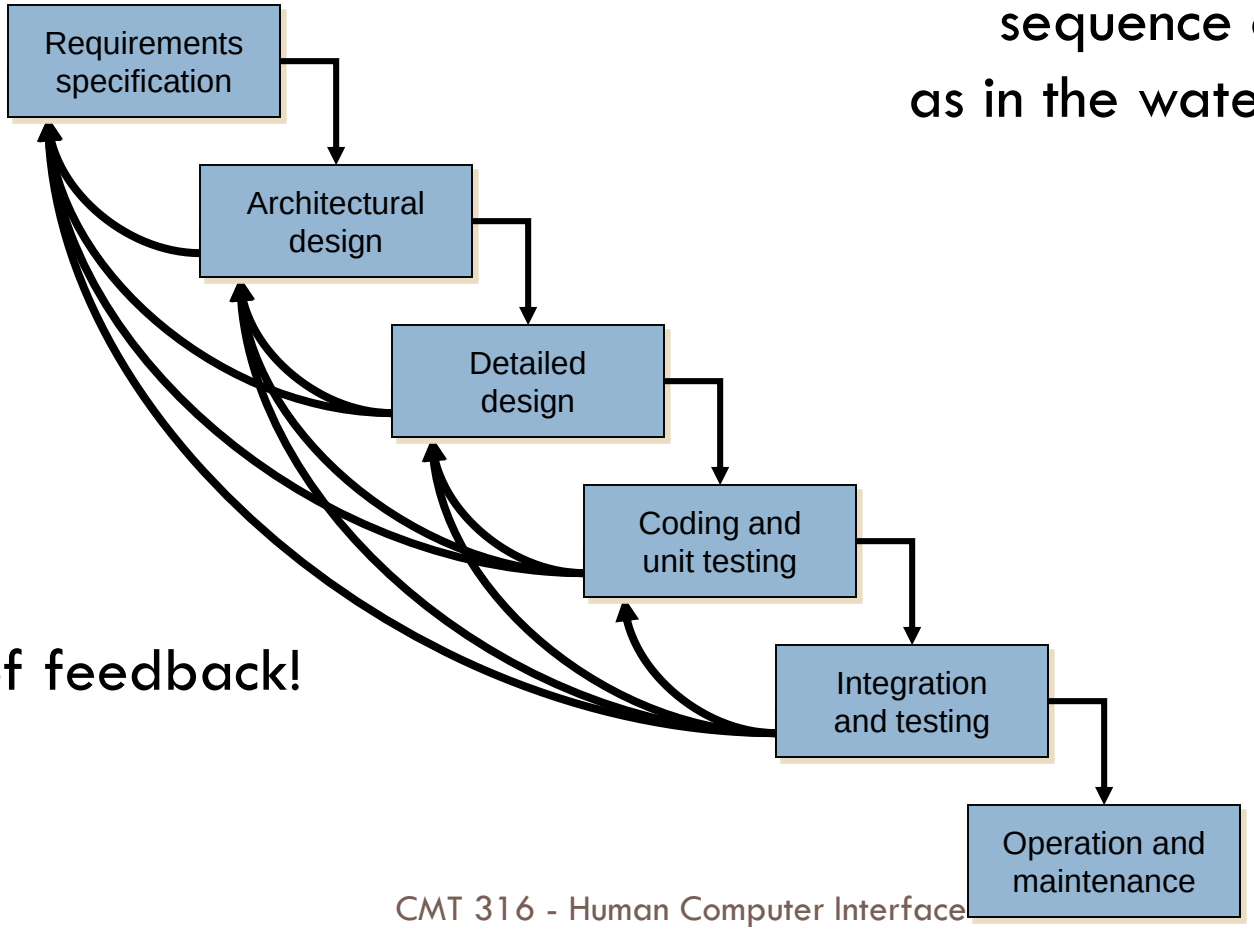
- validation will always rely to some extent on subjective means of proof

Management and contractual issues

- design in commercial and legal contexts

The life cycle for interactive systems

cannot assume a linear sequence of activities as in the waterfall model



lots of feedback!

HCI life-cycle

- Design
  - Process of developing artefacts
  - Various representations of artefacts during design
  - Look at *user-centered* design
- Development
  - Producing range of representations until suitable artefacts is built
- Representations (models): formal + informal
  - Users
  - Model of users
  - Look at several models for the design process

S/W Engineering HCI

- Identifies issues (entities) of significance to system
- Functionally oriented, data driven
- Design notation: to be understood by system analysts
  - Dataflow diagrams
  - Entity Relationship Diagrams
  - Resistance by future users

- Identifies issues (entities) of practical effectiveness
- Usability oriented, user driven
- Design notation: to be understood by users
  - Task Analysis (diagrams)
  - Scenarios
  - Screen sketch
  - Prototype

S/W Engineering and HCI

- **HCI Engineering and Software Engineering are keepers of different views.**

HCI Engineering (keeper of the user view)

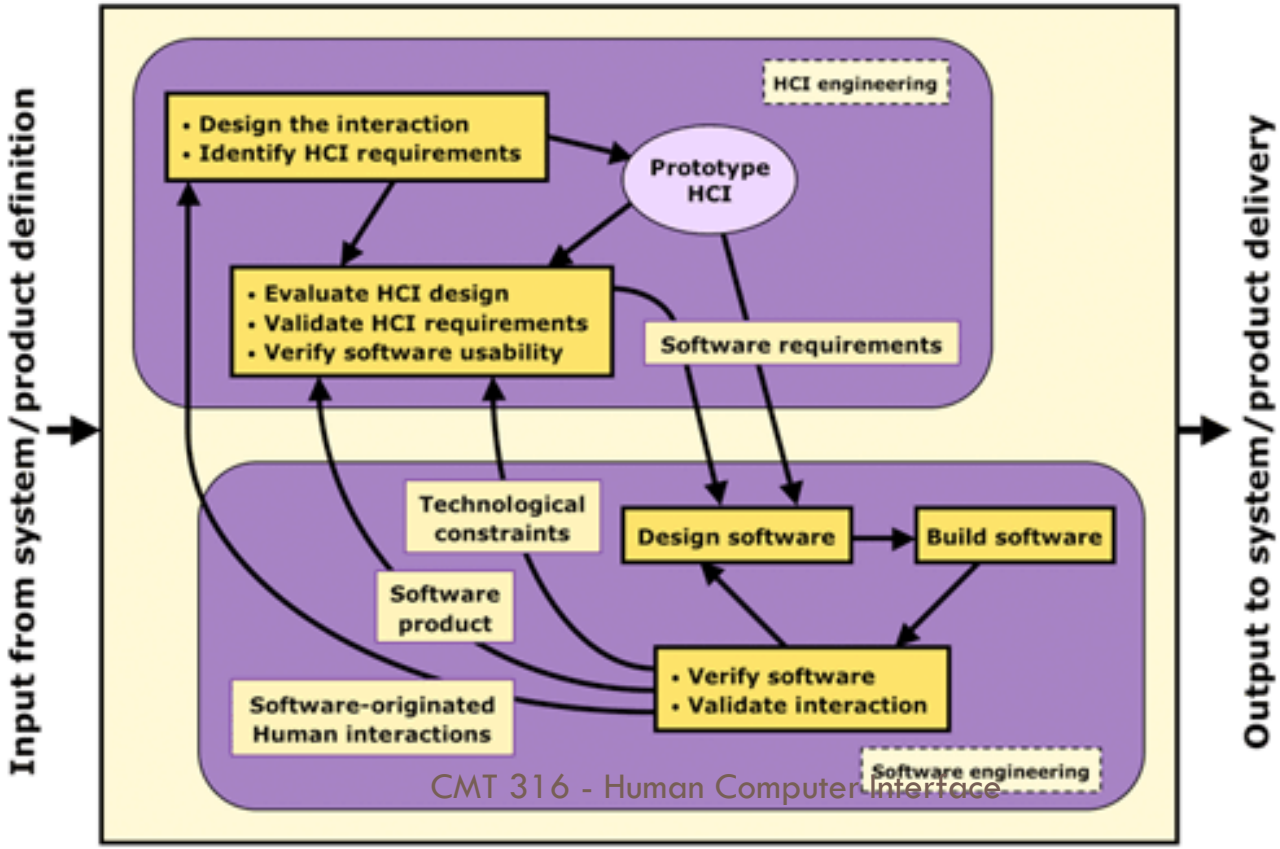
- User task analysis
- Usability specification
- Interaction architecture/design
- Interface design/specification
- Software support identification
- Usability evaluation

Software Engineering (keeper of the technology view)

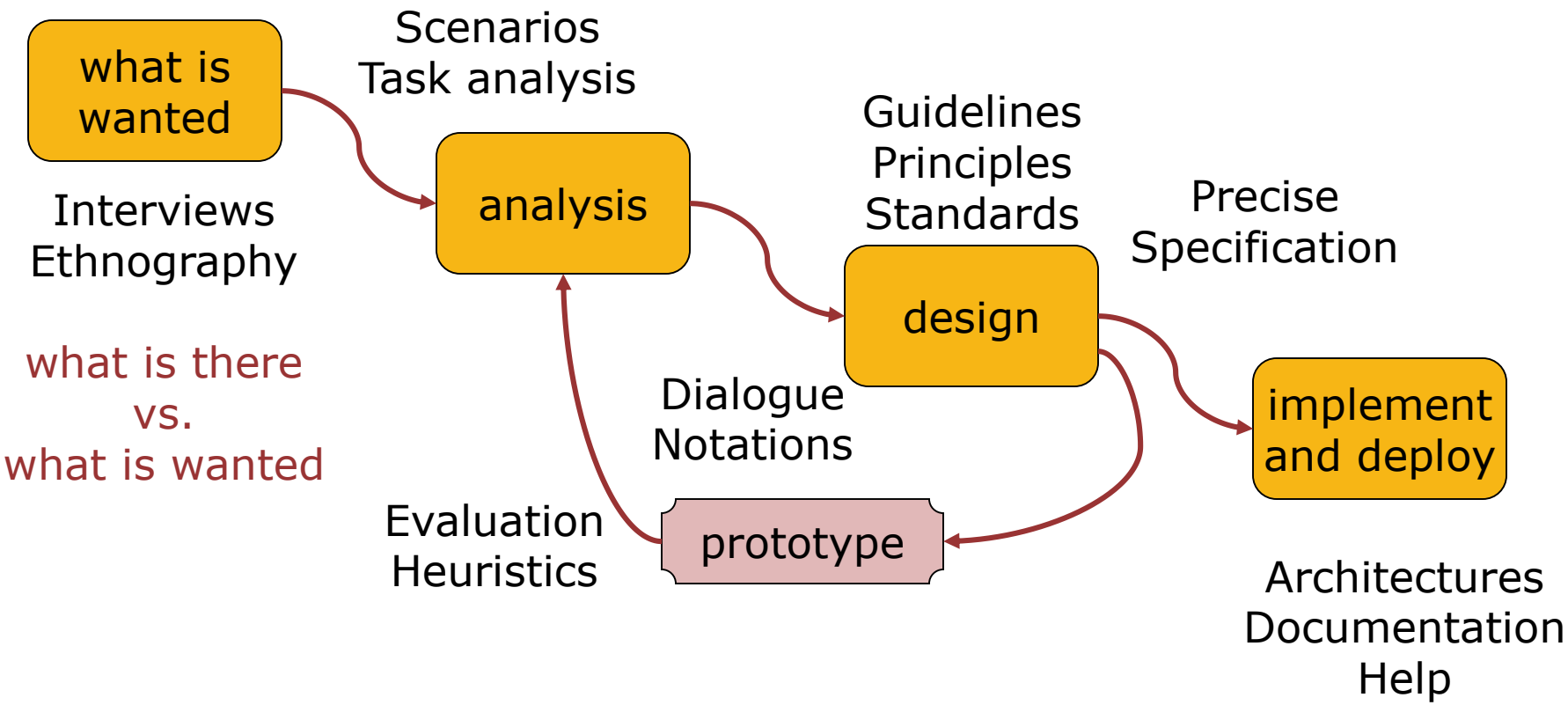
- Software requirements analysis
- Software design
- Interface software coding
- Interaction support software coding
- Application program coding
- Software verification/validation

S/W Engineering and HCI

- **System Engineering Coordinates the Activities of HCI Engineering and Software Engineering**



The process of design (Recap)



USABILITY ENGINEERING

Software Quality (ISO 9126)

- Metrics and Evaluation
  - Functionality
  - Reliability
  - Usability
  - Efficiency
  - Maintainability
  - Portability
  - PM Accessibility *(sometimes relevant, not ISO 9126)*

Definition of Usability (Nielsen 2003)

- Usability: how well users can use the system's functionality
- Usability is a quality attribute that assesses how easy user interfaces are to use
- The word 'usability' also refers to methods for improving ease-of-use during the design process

Usability Components

- Learnability
  - Ease of learning the system, i.e. the basic tasks
  - Skills retained over time (also Memorability)
- Throughput (also Efficiency)
  - Speed of task performance
  - Low user error rate
- Flexibility
  - Suitability for intended user expertise
  - Can system be customised?
- Attitude (also Satisfaction)
  - User subjective satisfaction with system

The 5E's to Usability

- Easy to learn
  - Supports initial learning and continued learning
- Efficient
  - The speed in which users complete their tasks
- Error tolerant
  - The ability of the interface to prevent errors or help users recover from those that occur
- Effective
  - The completeness and accuracy with which users achieve their goals
- Engaging
  - How pleasant or satisfying the interface is to use?

Usability Components (Dimensions of usability)

- Learnability: is it easy to learn?
- Memorability: is it easy to remember what you learned?
- Efficiency: once learned, is it fast to use?
- Errors: are errors few and recoverable?
- Satisfaction: is it enjoyable to use?

Usability Engineering

- The test of usability based on measurement of user experience
- It demands that specific usability measures be made explicit as requirements
- Usability Measurements
  - Usability attribute/principle
  - Measuring concept
  - Measuring method
  - Now level/ worst case/ planned level/ best case

Some metrics from ISO 9241

Usability objective	Effectiveness measures	Efficiency measures	Satisfaction measures
Suitability for the task	Percentage of goals achieved	Time to complete a task	Rating scale for satisfaction
Appropriate for trained users	Number of power features used	Relative efficiency compared with an expert user	Rating scale for satisfaction with power features
Learnability	% of functions learned	Time to learn criterion	Rating scale for ease of learning
Error tolerance	% of errors corrected successfully	Time spent on correcting errors	Rating scale for error handling

Usability Engineering (contd.)

- Problems
  - usability specification requires level of detail that may not be possible early in design satisfying a usability specification
  - does not necessarily satisfy usability

Case Study-ATM (from L9)

- Researchers + banks expected ATMs to be intuitively easy to use
- Testing among senior citizens found only 20% correct operation
- Senior citizens often put off by ATMs they find the machines
  - Complicated
  - Inconvenient

Problems with ATMs

- Buttons that did not line up with commands

- Screens hard to read in the glare of daylight

- Sometimes confusing menu choices

Usability suggestions

- Simpler on-screen instructions
- More “undo” buttons
- Bank should provide training for any customers who need it
- Remark:
  - “Large percentage” of people surveyed said they would use ATMs if trained

PROTOTYPING

Prototyping

- Prototyping is common in almost every field of engineering design
- In prototyping one has to plan to throw away

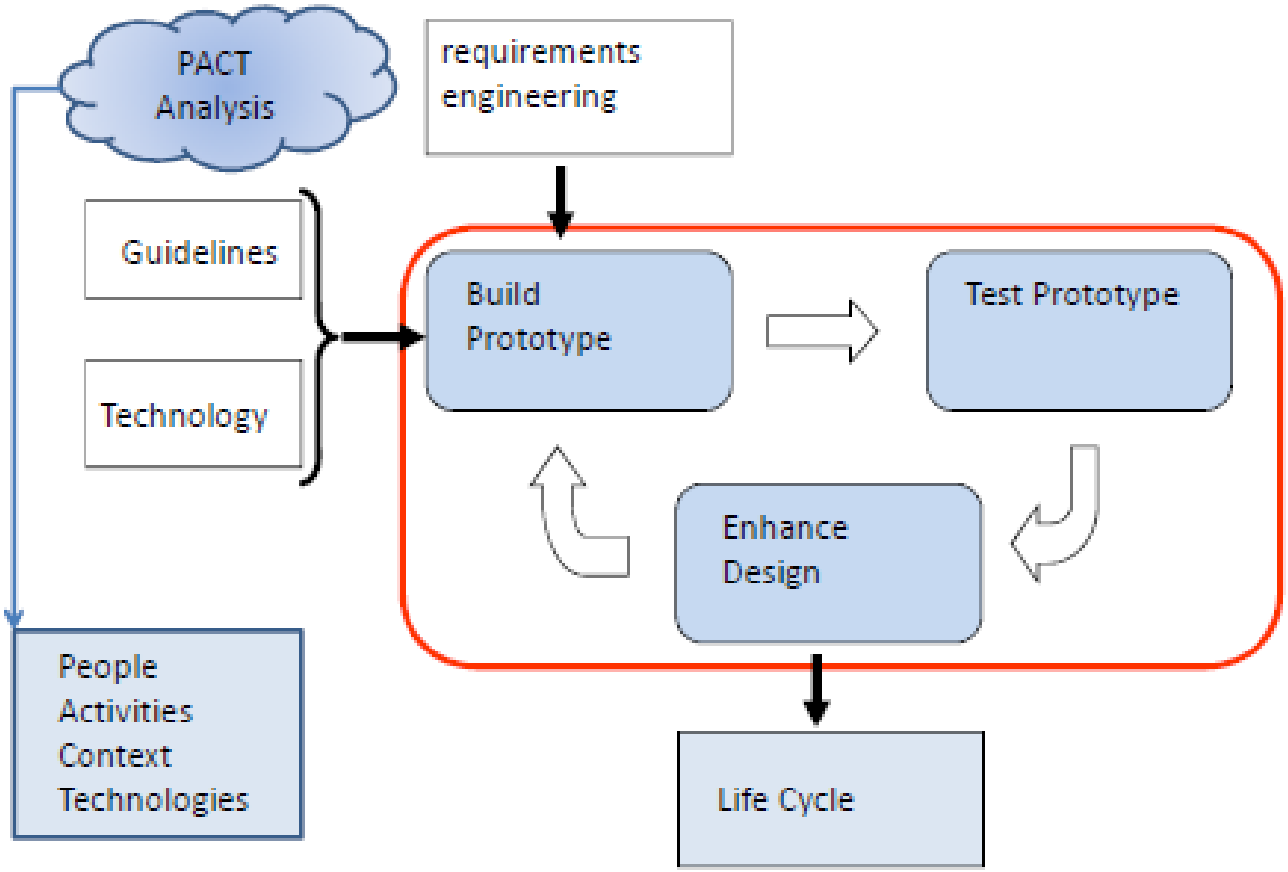
Why prototype?

- Get feedback earlier, cheaper
- Experiment with alternatives
- Easier to change to throw away

Prototyping the User-Interface

- Identifying user interface requirements
  - Almost impossible to specify in advance
- Supported by good software tools
  - Graphical editors, construction kits
  - User Interface Management Systems (UIMS)
  - Wireframes
- "Prototyping does *NOT* mean '*build in haste*'"

Prototyping the user interface



Prototype Fidelity

- An essential property of a prototyping technique is its **fidelity**
- It is simply express how similar it is to the finished interface or
- It refers to the level of detail
- Low fidelity
  - Omits details
- High fidelity
  - More like finished product

Summary

- Prototype fidelity
  - Depth, breadth, look, feel
- Kinds of prototypes
  - Paper
  - Computer: storyboard, forms
  - Wizard of Oz
- Don't get attached to a prototype
  - Because it may need to be thrown away